Distributed Information Processing

7th Lecture

Eom, Hyeonsang (엄현상)
Department of Computer Science & Engineering
Seoul National University

©Copyrights 2026 Eom, Hyeonsang All Rights Reserved

Outline

- Processes
  - Multithreading
  - Virtualization
  - Clients
  - Servers
- Q&A

Thread Usage in Nondistributed Systems

Figure 3-1 on Page 72

Context switching as the result of IPC

Context switch requires changing the memory map in the MMU (Memory Management Unit) and flushing the TLB (Translation Lookaside Buffer)

Tanenbaum & Van Steen, Distributed Systems: Principles and Paradigms, 2e, (c) 2007 Prentice-Hall, Inc. All rights reserved

Thread Implementation

- User-Level Threads
  - Being Executed Entirely in User Space
    - It is cheap to create and destroy threads
    - Switching can often be done efficiently
    - Invocation of a blocking system call blocks all
  - Having the Kernel Aware of Threads and Schedule Them
    - Switching may become expensive
- Lightweight Process (LWP)
  - Hybrid Form of User and Kernel-Level Threads
    - Each LWP can run its own (user-level) thread
    - Synchronization does not require kernel support

Thread Implementation (Cont'd)

Figure 3-2 on Page 74

Combining kernel-level lightweight processes and user-level threads

Tread doing a blocking system call still continues in the context of the current LWP.
The OS may decide to switch to another LWP.
In switching, the selected LWP will simply continue where it had previously left off.
A blocking system call will not suspend the entire process having enough LWPs.

Tanenbaum & Van Steen, Distributed Systems: Principles and Paradigms, 2e, (c) 2007 Prentice-Hall, Inc. All rights reserved

Multithreaded Servers

Figure 3-3 on Page 77

A multithreaded server organized in a dispatcher/worker model

Tanenbaum & Van Steen, Distributed Systems: Principles and Paradigms, 2e, (c) 2007 Prentice-Hall, Inc. All rights reserved

Multithreaded Servers (Cont'd)

Figure 3-4 on Page 78

Finite-state Machine

The process is being operated as a finite state machine that gets an event and then react to it, depending on what is in it; this approach performs well, but is hard to program

Three ways to construct a server

Tanenbaum & Van Steen, Distributed Systems: Principles and Paradigms, 2e, (c) 2007 Prentice-Hall, Inc. All rights reserved

The Role of Virtualization in Distributed Systems

Figure 3-5 (a) on Page 80

Figure 3-5 (b) on Page 80

General organization between a program, interface, and system. (b) General organization of virtualizing system A on top of system B

Tanenbaum & Van Steen, Distributed Systems: Principles and Paradigms, 2e, (c) 2007 Prentice-Hall, Inc. All rights reserved

Architectures of Virtual Machines

- Interfaces at Different Levels
  - An interface between the hardware and software consisting of **machine instructions**
    - that can be invoked by any program
  - An interface between the hardware and software, consisting of machine instructions
    - that can be invoked only by privileged programs, such as an operating system

Tanenbaum & Van Steen, Distributed Systems: Principles and Paradigms, 2e, (c) 2007 Prentice-Hall, Inc. All rights reserved

Architectures of Virtual Machines (Cont'd))

- Interfaces at Different Levels (Cont'd)
  - An interface consisting of **system calls** as offered by an operating system
  - An interface consisting of **library calls**
    - generally forming what is known as an application programming interface (API)
    - In many cases, the aforementioned system calls are hidden by an API

Tanenbaum & Van Steen, Distributed Systems: Principles and Paradigms, 2e, (c) 2007 Prentice-Hall, Inc. All rights reserved

Architectures of Virtual Machines (Cont'd)

Figure 3-6 on Page 81

Various interfaces offered by computer systems

Tanenbaum & Van Steen, Distributed Systems: Principles and Paradigms, 2e, (c) 2007 Prentice-Hall, Inc. All rights reserved

Architectures of Virtual Machines (Cont'd)

Runtime system

Instructions can be interpreted (as is the case for the Java executing applications), but could also be emulated mimicking the behavior of system calls

Figure 3-7 (a) on Page 82

(a) A process virtual machine, with multiple instances of (application, runtime) combinations

Tanenbaum & Van Steen, Distributed Systems: Principles and Paradigms, 2e, (c) 2007 Prentice-Hall, Inc. All rights reserved

Architectures of Virtual Machines (Cont'd)

Figure 3-7 (b) on Page 82

(b) A virtual machine monitor, with multiple instances of (applications, operating system) combinations

Tanenbaum & Van Steen, Distributed Systems: Principles and Paradigms, 2e, (c) 2007 Prentice-Hall, Inc. All rights reserved

Virtualization

- Creating a virtual (rather than actual) version of something:
  - Virtual computer H/W
  - OS
  - Storage devices
  - Computer network resources
  - Others

[Wikipedia]

H/W Virtualization

- Creating a VM that acts like a real computer with an OS
  - Host machine
    - Actual machine on which the virtualization takes place
  - Guest machine
    - VM

[Wikipedia]

Clarification of Basic Terms

- Commonality between emulator & hypervisor
  - Computer programs that imitate H/W
- Differences between them
  - Different domains of use

Emulators duplicate the behavior of entirely different hardware architectures via software simulation, **while** hypervisors isolate and partition the same underlying physical hardware into VMs

[Wikipedia: Gemini]

Types of H/W Virtualization

- Full virtualization
  - Almost complete simulation of the actual H/W to allow S/W, which typically consists of a guest OS, to run unmodified
- Partial virtualization
  - Some but not all of the target environment is simulated
    - Some guest programs, thus, may need modifications

[Wikipedia]

Types of H/W Virtualization (Cont'd)

- Paravirtualization (PV)
  - H/W environment is not simulated, but the guest programs are executed in their own isolated domains as if they were running on separate systems after being modified

H/W-assisted virtualization (HVM): a way of improving overall efficiency of virtualization

[Wikipedia]

Full Virtualization

- Example of full virtualization
  - VMware employs binary translation to automatically modify x86 S/W on-the-fly to replace instructions
- Why full virtualization?
  - **Sharing** a computer system among multiple users
  - **Isolating** users from each other (and from the control program)
  - **Emulating** new H/W to achieve improved reliability, security and productivity

[Wikipedia]

Hypervisor

- VMM (Virtual Machine Monitor)
  - Piece of computer S/W, F/W or H/W that creates and runs VMs
- Host Machine
  - Computer on which a hypervisor is running one or more VMs
- Guest Machine
  - Each VM

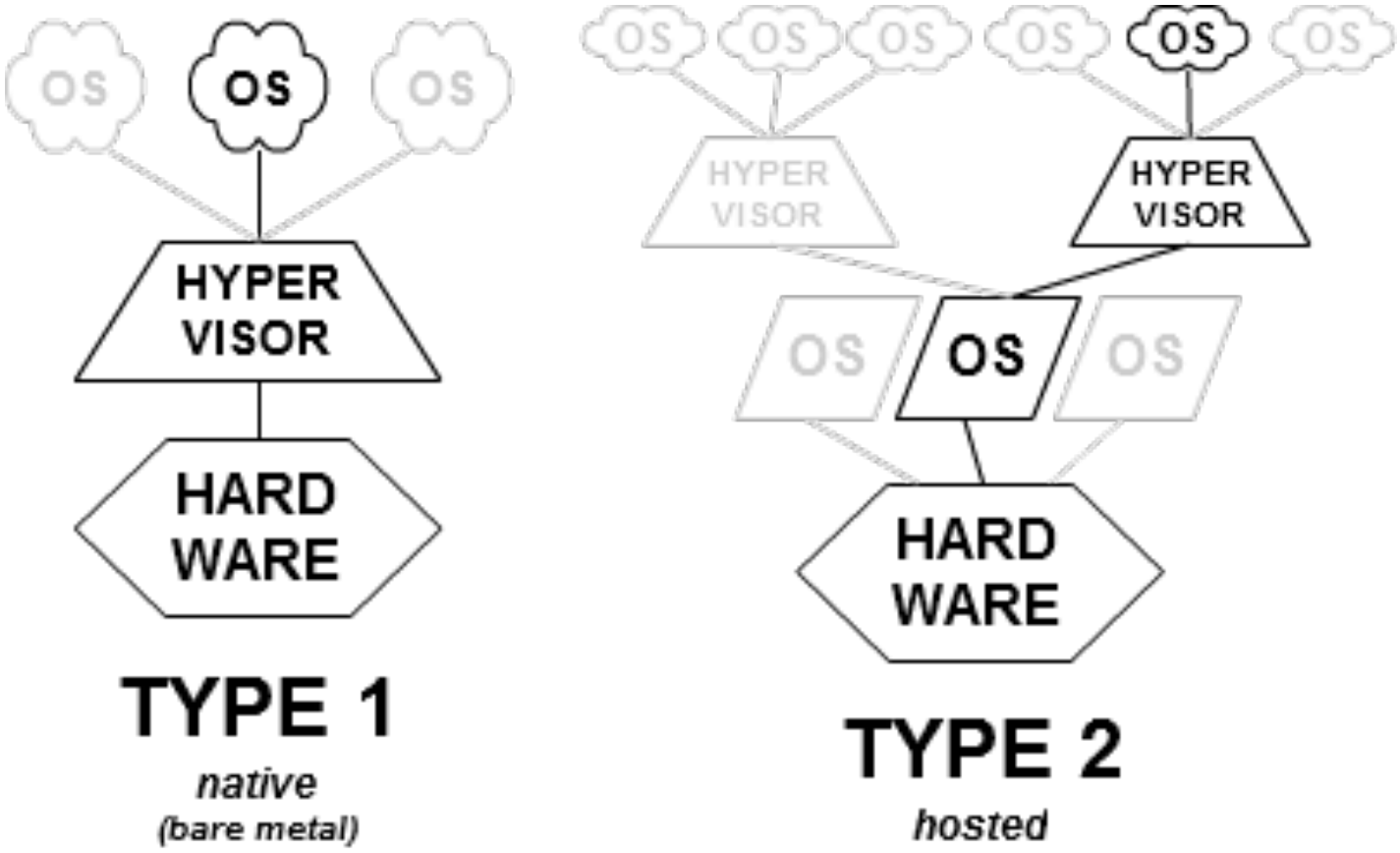
[Wikipedia]

Types of Hypervisor

- Type-1: Native or bare-metal hypervisors
  - Running directly on the host's H/W to control H/W and manage guest OSes
    - E.g., VMware ESX/ESXi
- Type-2: Hosted hypervisors
  - Running on a conventional OS
- Both: e.g., Linux's KVM (Kernel-based VM)
  - Effectively converting the host OS to a Type-1
  - Linux still being a general purpose OS

[Wikipedia]

Illustration: Types of Hypervisor



[Wikipedia]

KVM & QEMU (Quick Emulator)

- KVM by itself only handles CPU and memory virtualization
  - It does not know how to emulate hardware peripherals like disk controllers, USB ports, or network cards
- QEMU acts as the user-space management tool
  - It emulates the rest of the virtual hardware and handles the VM's configuration

[Wikipedia; Gemini]

KVM & QEMU Architecture

- User Space (Management & Emulation)
  - Guest OS
  - QEMU
- Kernel Space (The Engine)
  - KVM Kernel Module: Type-1 hypervisor
  - Linux Subsystems
- **Hardware Layer (The Accelerator)**
  - Intel VT-x / AMD-V
  - VM-Exit / VM-Entry

[Wikipedia: Gemini]

Networked User Interfaces

Figure 3-8 (a) on Page 83

(a) A networked application with its own protocol

Tanenbaum & Van Steen, Distributed Systems: Principles and Paradigms, 2e, (c) 2007 Prentice-Hall, Inc. All rights reserved

Networked User Interfaces (Cont'd)

Thin-Client Approach

Figure 3-8 (b) on Page 83

(b) A general solution to allow access to remote applications

Tanenbaum & Van Steen, Distributed Systems: Principles and Paradigms, 2e, (c) 2007 Prentice-Hall, Inc. All rights reserved

Example: The X Window System

Figure 3-9 on Page 84

The basic organization of the X Window System

Tanenbaum & Van Steen, Distributed Systems: Principles and Paradigms, 2e, (c) 2007 Prentice-Hall, Inc. All rights reserved

Client-Side Software for Distribution Transparency

Figure 3-10 on Page 88

Transparent replication of a server using a client-side solution

Tanenbaum & Van Steen, Distributed Systems: Principles and Paradigms, 2e, (c) 2007 Prentice-Hall, Inc. All rights reserved

General Design Issues

Figure 3-11 (a) on Page 90

(a) Client-to-server binding using a daemon

Tanenbaum & Van Steen, Distributed Systems: Principles and Paradigms, 2e, (c) 2007 Prentice-Hall, Inc. All rights reserved

General Design Issues (Cont'd)

Figure 3-11 (b) on Page 90

(b) Client-to-server binding using a superserver

Tanenbaum & Van Steen, Distributed Systems: Principles and Paradigms, 2e, (c) 2007 Prentice-Hall, Inc. All rights reserved

Server Clusters

Figure 3-12 on Page 93

The general organization of a three-tiered server cluster

Tanenbaum & Van Steen, Distributed Systems: Principles and Paradigms, 2e, (c) 2007 Prentice-Hall, Inc. All rights reserved

Server Clusters (Cont'd)

Figure 3-13 on Page 95

The principle of TCP handoff

Tanenbaum & Van Steen, Distributed Systems: Principles and Paradigms, 2e, (c) 2007 Prentice-Hall, Inc. All rights reserved

Distributed Servers

Figure 3-14 on Page 97

Route optimization in a distributed server

Tanenbaum & Van Steen, Distributed Systems: Principles and Paradigms, 2e, (c) 2007 Prentice-Hall, Inc. All rights reserved